

K-Means and K-Nearest Neighbour (Semi-Supervised Learning model)

Student id: C00313459

Name: Amisha Das

1. Introduction

This work is incomplete. It is an attempt at a semi-supervised learning model (SSL) combining the supervised K Nearest Neighbour (KNN) and unsupervised K-Means models.

```
In [1]: import pandas as pd
        from sklearn.cluster import KMeans
        from sklearn.neighbors import KNeighborsRegressor
        from sklearn.preprocessing import StandardScaler
        import numpy as np
```

```
In [2]: df = pd.read_csv("sdss_star_classification.csv")
```

```
In [3]: features = ['u', 'g', 'r', 'i', 'z', 'alpha', 'delta']
        target = 'redshift'
```

splitting data into labelled and unlabelled

```
In [4]: labeled_data = df[df[target].notnull()].copy()
        unlabeled_data = df[df[target].isnull()].copy()
```

```
In [5]: scaler = StandardScaler()
        labeled_features_scaled = scaler.fit_transform(labeled_data[features])
```

K MEANS

```
In [6]: kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
        labeled_data['cluster'] = kmeans.fit_predict(labeled_features_scaled)
```

KNN

```
In [7]: knn = KNeighborsRegressor(n_neighbors=5)
        knn.fit(labeled_features_scaled, labeled_data[target])
```

```
Out[7]: KNeighborsRegressor
        KNeighborsRegressor()
```

Predicting Redshift

```
In [8]: if not unlabeled_data.empty:
        unlabeled_features_scaled = scaler.transform(unlabeled_data[features])
        pseudo_labels = knn.predict(unlabeled_features_scaled)
        unlabeled_data['pseudo_redshift'] = pseudo_labels
        else:
        unlabeled_features_scaled = None
```

```
In [9]: labeled_data.to_csv("clustered_labeled_data.csv", index=False)
        if not unlabeled_data.empty:
        unlabeled_data.to_csv("pseudo_labeled_data.csv", index=False)
```

Evaluating (incomplete)

silhouette score

```
In [10]: from sklearn.metrics import silhouette_score, mean_absolute_error, mean_squared_error
```

```
In [11]: silhouette_avg = silhouette_score(labeled_features_scaled, labeled_data['cluster'])
        print(f"Silhouette Score for K-Means clustering: {silhouette_avg:.4f}")
```

Silhouette Score for K-Means clustering: 0.3459

MAE and MSE for KNN

```
In [12]: knn_predictions = knn.predict(labeled_features_scaled)
mae = mean_absolute_error(labeled_data[target], knn_predictions)
mse = mean_squared_error(labeled_data[target], knn_predictions)
print(f"KNN Mean Absolute Error (MAE): {mae:.6f}")
print(f"KNN Mean Squared Error (MSE): {mse:.6f}")
```

KNN Mean Absolute Error (MAE): 0.259812
KNN Mean Squared Error (MSE): 0.264545

Lower value is a sign of good prediction.

```
In [13]: labeled_data.to_csv("clustered_labeled_data.csv", index=False)
if not unlabeled_data.empty:
    unlabeled_data.to_csv("pseudo_labeled_data.csv", index=False)
```

Creating artificially unlabeled data for trsting pseudolabels

```
In [14]: df.loc[df.sample(frac=0.2, random_state=42).index, 'redshift'] = np.nan
```

Log

```
In [16]: log = pd.read_csv("log.csv")
pd.set_option('display.max_colwidth', None)
log
```

Out[16]:

	Model	Description	Level of Difficulty	Time Spent	Change From	Changed To	Notes
0	KNN	Imported mean_absolute_error, mean_squared_error	Easy	10 mins	NaN	Added imports for error measurement	NaN
1	KNN	Used StandardScaler for feature normalization	Medium	15 min	NaN	Applied StandardScaler to dataset	NaN
2	Data	Split data into labeled and unlabeled	Medium	20 min	No unlabeled data available	Manually removed labels from some data	NaN
3	K-Means	Standardized features, chose k=5 for K-Means	Medium	30 mins	NaN	Set k=5 for initial testing	NaN
4	KNN	Trained KNN on labeled dataset	Hard	40 min	NaN	KNN trained using labeled subset	NaN

Future Work

1. add confidence based pseudo-labelling using knn distance
2. make changes to standardisation of unlabeled data before pseudod-labelling
3. and PCA for pseuo-labelling

```
In [ ]:
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js